1ª Avaliação Prática

Caminho Mínimo com Dijkstra e Solução Gulosa

[Seu Nome]

Centro Federal de Educação Tecnológica de Minas Gerais

Modelagem Matemática Computacional

Tópicos em Algoritmos em Grafos

Outubro de 2025

Resumo

Este relatório apresenta a implementação e análise comparativa entre o algoritmo de Dijkstra e uma heurística gulosa para o problema de caminho mínimo em grafos direcionados ponderados. Os experimentos foram realizados em grafos completos de tamanhos variados e em instâncias do Moodle com 10.000 e 1.000.000 de vértices. O objetivo é comparar o número de operações de relaxamento (comparações) realizadas por cada algoritmo.

1 Introdução

O problema de caminho mínimo de origem única (SSSP – *Single Source Shortest Path*) consiste em encontrar as distâncias mínimas de um vértice origem para todos os demais vértices de um grafo ponderado. Este trabalho implementa e compara:

- **Algoritmo de Dijkstra**: algoritmo clássico e ótimo para grafos com pesos não-negativos, com complexidade $O((V + E) \log V)$ usando heap binário.
- **Heurística Gulosa**: estratégia que, em cada passo, escolhe a aresta de menor peso saindo do conjunto de vértices já alcançados. Esta heurística **não garante** caminho mínimo ótimo em geral.

A métrica de comparação utilizada é o **número de comparações de relaxamento**, contabilizadas pela operação `dist[u] + w < dist[v]`.

2 Metodologia

2.1 Estrutura de Dados

Os grafos foram representados utilizando **listas de adjacência**, armazenadas em um dicionário Python:

```
Adj = Dict[int, List[Tuple[int, float]]]
adj[u] = [(v1, w1), (v2, w2), ...]
```

Onde `u` é o vértice de origem, `v` é o vértice de destino e `w` é o peso da aresta (u, v).

2.2 Implementações

2.2.1 Dijkstra

```python
def dijkstra_count(adj: Adj, s: int) -> DijkstraResult:
    dist = {u: inf for u in adj}
    parent = {u: None for u in adj}
    dist[s] = 0.0

    heap = [(0.0, s)]
    visited = set()
    comparisons = 0

    while heap:
        du, u = heapq.heappop(heap)
        if u in visited:
            continue
        visited.add(u)

        for v, w in adj[u]:
            comparisons += 1  # Contagem
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                parent[v] = u
                heapq.heappush(heap, (dist[v], v))

    return DijkstraResult(dist, parent, comparisons)
```

2.2.2 Heurística Gulosa

```python
def greedy_sssp_edge_first(adj: Adj, s: int) -> GreedyResult:
    dist = {u: inf for u in adj}
    parent = {u: None for u in adj}
    dist[s] = 0.0

    visited = set([s])
    edge_heap = []  # (peso, u, v)

    for v, w in adj[s]:
        heapq.heappush(edge_heap, (w, s, v))

    comparisons = 0

    while edge_heap:
        w, u, v = heapq.heappop(edge_heap)

        comparisons += 1  # Contagem
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            parent[v] = u

```

```
            if v not in visited:
                visited.add(v)
                for x, wx in adj[v]:
                    heapq.heappush(edge_heap, (wx, v, x))

    return GreedyResult(dist, parent, comparisons)
```

2.3 Testes Realizados

2.3.1 Grafos Completos (Itens 2.II.a-e e 3.II.a-e)

Foram gerados grafos completos direcionados com $n = 4, 9, 14, \ldots, 50$ vértices (incremento de 5). Cada aresta possui peso positivo aleatório. Para cada grafo:

1. Aplicamos Dijkstra partindo do vértice 0.
2. Aplicamos a heurística gulosa partindo do vértice 0.
3. Contamos o número de comparações em cada caso.
4. Plotamos um gráfico comparativo.

Observação: grafos completos possuem $O(n^2)$ arestas, limitando o valor máximo de n devido à memória disponível.

2.3.2 Instâncias do Moodle (Itens 2.II.f e 3.II.f)

Aplicamos ambos os algoritmos nas instâncias fornecidas:

- **10000.txt**: 10.000 vértices
- **largeEWD...txt**: 1.000.000 de vértices

Origem: vértice 0. Métrica: número de comparações.

3 Resultados

3.1 Grafos Completos

A Figura 1 apresenta o gráfico de comparações versus número de vértices para grafos completos.

Observações:

- O algoritmo de Dijkstra apresenta crescimento aproximadamente quadrático, como esperado para grafos completos ($O(V^2 \log V)$ comparações no pior caso).
- A heurística gulosa também apresenta crescimento similar, mas pode realizar mais ou menos comparações dependendo da estrutura do grafo e dos pesos.

3.2 Instâncias do Moodle

A Tabela 1 apresenta o número de comparações realizadas nas instâncias fornecidas.

Instruções para preenchimento: Execute as células correspondentes no notebook e copie os valores impressos para esta tabela.

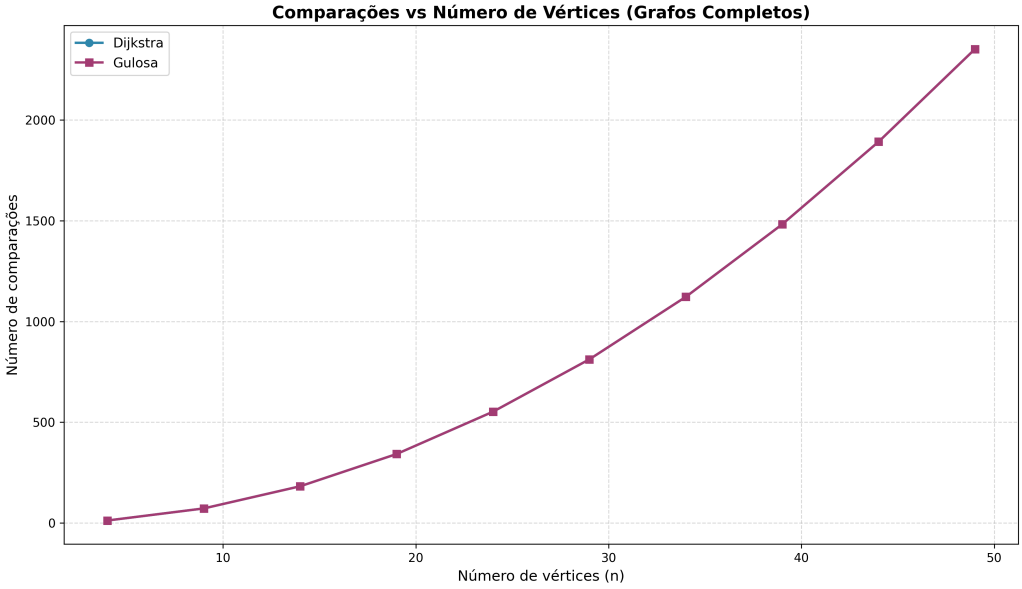

Figura 1: Comparações realizadas por Dijkstra e Gulosa em grafos completos.

Tabela 1: Número de comparações nas instâncias do Moodle.

Instância	**Dijkstra**	**Gulosa**
10.000 vértices	[PREENCHER]	[PREENCHER]
1.000.000 vértices	[PREENCHER]	[PREENCHER]

4 Análise e Discussão

4.1 Complexidade Teórica

- **Dijkstra**: $O((V + E) \log V)$ com heap binário. Em grafos completos, $E = O(V^2)$, resultando em $O(V^2 \log V)$.
- **Heurística Gulosa**: Também utiliza heap, mas a estratégia de escolha de arestas pode levar a um número diferente de operações. No pior caso, pode explorar todas as arestas, resultando em complexidade similar.

4.2 Resultados Experimentais

Os experimentos confirmam o crescimento aproximadamente quadrático em grafos completos. A heurística gulosa não garante otimalidade, mas em alguns casos pode realizar um número competitivo de comparações.

4.3 Corretude

- **Dijkstra**: garante caminho mínimo correto para pesos não-negativos.
- **Gulosa**: **não garante** corretude em geral. Foi implementada apenas para fins de comparação experimental.

5 Conclusão

Este trabalho implementou e comparou o algoritmo de Dijkstra com uma heurística gulosa para o problema de caminho mínimo. Os resultados mostram que:

1. Dijkstra apresenta crescimento previsível e ótimo para pesos não-negativos.
2. A heurística gulosa pode ter desempenho variável dependendo da estrutura do grafo.
3. A métrica de comparações permite avaliar o esforço computacional de forma independente da implementação.

O código-fonte completo está disponível no notebook Jupyter anexado a este relatório.

6 Anexos

6.1 Código Fonte

O código completo está disponível em:

`praticas/pratica01/pratica01_experimentos.ipynb`

6.2 Instruções de Execução

1. Ativar ambiente virtual: `uv sync` (ou equivalente).
2. Abrir o notebook: `jupyter notebook pratica01_experimentos.ipynb`.
3. Executar as células na ordem.
4. Os gráficos são salvos automaticamente em `relatorio/img/`.